Question 1: Create and Use a Named Volume

Objective: Understand how to create and mount a named volume.

Task:

1. Create a named volume called `app-data`.
2. Run an `alpine` container that mounts this volume to `/data` inside the container.
3. Inside the container, create a file `message.txt` with the text `"Hello from volume!"` inside `/data`.
4. Exit the container.
5. Run a **new** container (using the same volume) and verify the file still exists.

Hints:

- Create volume: `docker volume create app-data`
- Run container with mount: `docker run -it --rm -v app-data:/data alpine sh`
- Use `echo "Hello from volume!" > /data/message.txt`
- Run second container: `docker run --rm -v app-data:/data alpine cat /data/message.txt`

Question 2: Bind Mount (Host Directory as Volume)

Objective: Use a host directory as a volume for live development.

Task:

1. Create a directory on your host machine called `my-website`.
2. Create an `index.html` file inside it with basic HTML content (e.g., `<h1>My Docker Site</h1>`).
3. Run an `nginx` container that mounts this directory to `/usr/share/nginx/html` inside the container.
4. Map port `8080` on your host to port `80` in the container.
5. Access `http://localhost:8080` in your browser and verify you see your HTML page.
6. **Without stopping the container**, edit the `index.html` file on your host and refresh the browser – the change should reflect immediately.

Hints:

- Create directory: `mkdir my-website`

- Create file: `echo "<h1>My Docker Site</h1>" > my-website/index.html`
- Run: `docker run -d -p 8080:80 -v $(pwd)/my-website:/usr/share/nginx/html --name web nginx`
- Use `$(pwd)` on Linux/Mac or full path on Windows.

Question 3: Share Data Between Two Containers Using a Volume

Objective: Demonstrate data persistence and sharing across containers.

Task:

1. Create a volume named `shared-data`.
2. Run **Container A** (`alpine`) that writes the current date and time to a file `timestamp.txt` inside the volume (mounted to `/shared`).
3. Exit Container A.
4. Run **Container B** (another `alpine` container) that reads and displays the content of `timestamp.txt` from the same volume.
5. Run Container A again to append a new timestamp to the file.
6. Run Container B again to show both timestamps.

Hints:

- Create volume: `docker volume create shared-data`
- Container A (write): `docker run --rm -v shared-data:/shared alpine sh -c "date >> /shared/timestamp.txt"`
- Container B (read): `docker run --rm -v shared-data:/shared alpine cat /shared/timestamp.txt`
- Run the write command twice, read after each write.

Question 4: Volume Cleanup and Inspection

Objective: Learn to manage and clean up volumes.

Task:

1. Create **three** named volumes: `vol1`, `vol2`, `vol3`.

2. List all volumes and inspect one of them to see its details (mountpoint, driver, etc.).
3. Remove `vol1` and `vol2` individually.
4. Use a single command to remove **all unused** volumes (including `vol3` if it's not being used).
5. Verify that no volumes remain.

Hints:

- Create: `docker volume create vol1` (repeat for vol2, vol3)
- List: `docker volume ls`
- Inspect: `docker volume inspect vol1`
- Remove individual: `docker volume rm vol1`
- Remove all unused: `docker volume prune` (confirm with `y`)
- Verify: `docker volume ls`

Bonus (Optional Challenge): Volume with Database Persistence

Task: Run a MySQL container with a named volume for database storage. Stop and remove the container, then create a new MySQL container using the same volume and verify that the data persists.

Hints:

- Run: `docker run -d --name mysql-db -v mysql-data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=mysecretpw mysql:latest`
- Connect, create a test database/table.
- Stop and remove: `docker stop mysql-db && docker rm mysql-db`
- Run new container with same volume: `docker run -d --name mysql-db-new -v mysql-data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=mysecretpw mysql:latest`
- Verify your database still exists.